

Forsøk 1



Pågår

NESTE: Lever oppgave



Legg til kommentar

Ubegrenset antall forsøk tillatt

▼ Detaljer

Guidelines

To get credit for this exercise, you must:

- **Hand in the exercise on time—delayed hand ins will not receive any credit.**
- Hand in the Jupyter notebook file (*.ipynb) and the same file exported as PDF. If PDF export fails, hand in the IPython notebook. Without the PDF, I will not be able to give you feedback directly in the document.
- **You must include your names and exercise group number at the top of the notebook:**
 - **No name, no credit!**
 - **One point is deducted if the group number is missing.**
- Place the exercises in the order given below in the notebook.
- Begin each exercise with a second-level heading (##) reading "Exercise X".
- Make reasonable attempts at solving all exercises.

For an introduction to how to typeset mathematics in notebooks, see, e.g., the [LaTeX section](#)

(<https://patrickwalls.github.io/mathematicalpython/jupyter/latex/>) of [Mathematical Python](#)

(<https://patrickwalls.github.io/mathematicalpython/>). You are also encouraged to look at the Markdown and LaTeX code in the lecture notebooks.

Exercise 1 (2 points)

Use the insertion sort algorithm presented in the lecture (pseudocode) to sort the array $A = [8, 9, 2, 3, 7, 1]$. Illustrate the progress of the pseudocode algorithm from the lecture using a table. The table shall have one row for each repetition of the for loop spanning rows 2–8 and shall show the values of j , i , key and A for each repetition immediately after line 4 has been executed; a final line in the table shall show A after the algorithm has terminated.

Here is an example for sorting $A = [c, a, b]$:

j	i	key	A
2	1	a	[c, a, b]
3	2	b	[a, c, b]
Terminated			[a, b, c]

Note: use 1-based indexing as we do for pseudocode.

Exercise 2 (3 points)

Consider the searching problem:

Input: A sequence of n numbers $A = \langle a_1, a_2, \dots, a_n \rangle$ and a value v

Output: An index i such that $v = A[i]$ or the special value NIL if v does not appear in A

Write pseudocode for a linear search which scans through the sequence looking for v .

What are the best, average and worst case run times of the algorithm for array size n ?

Exercise 3 (4 points)

We often need to sort collections of strings, for examples names in a phonebook. Advanced sorting algorithms will often convert all the strings to numbers first (each string to a different number) and then sort by comparing those numbers (known as sort

keys).

1. Why would it be an advantage to use numbers as sort keys instead of sorting the strings directly?
2. Consider the method used to convert strings to numbers: which condition must the numbers created fulfill?
3. Can you suggest an algorithm for converting strings into numbers to be used as sort keys? Assume that each letter in a string is 1 byte long, that numbers are 8 bytes long and that all strings are at most 8 letters long.

Exercise 4 (2 points)

Assume that for inputs of size n , the runtime for insertion sort is $T_I(n) = 16n^2$ and for merge sort $T_M(n) = 512n \lg n$. For which values of n is insertion sort faster than merge sort?

Exercise 5 (1 point)

Explain why the statement that the running time of an algorithm is at least $O(n^2)$ is meaningless.

Exercise 6 (3 points)

Express the function $T(n) = 2n^3 - 5\sqrt{n} + 4n \lg n$ in Θ -notation and give suitable constants .

Exercise 7 (5 points)

Consider the "find the smallest element" problem:

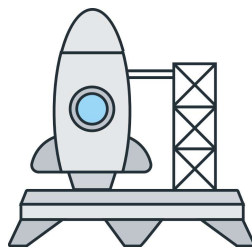
- **Input:** A sequence of n elements $\langle a_1, a_2, \dots, a_n \rangle$ for which a strict ordering is defined (i.e., $a_i < a_j$ is defined for all $1 \leq i, j \leq n$).
 - **Output:** a_i such that $a_i \leq a_k$ for all $1 \leq k \leq n$ for $n \geq 1$, otherwise NIL.
1. Develop a recursive algorithm to solve this task.
 2. Draw a recursion tree for the algorithm and try to estimate the run time $T(n)$ based on it.
 3. Try to find a recurrence equation for $T(n)$.
 4. Implement your recursive algorithm in Python and compare its runtime to the loop-based algorithm from the lab session in week 6 using `%timeit`.

Velg innsendelsestype

Last opp

⋮

Mer



Velg en fil å laste opp

Tillatt fil: PDF, IPYNB

eller

 Canvas-filer



<https://nmbu.instructure.com/courses/13608/modules/items/385946>



Oppgave 1

<https://nmbu.instructure.com/courses/13608/modules/item>